Chapters 9, 10, 11

PHP Advanced: File Handling, Classes & Objects, Working with Database

Objectives

1 $_GET and $_POST Superglobals

2 $_SERVER Array

3 PHP Form Handling

4 PHP File Handling

5 PHP OOP - Classes and Objects

6 PHP MySQL Database

PHP Include Files

- **The include (or require)** statement takes all the text/code/markup that exists in the specified file and copies it into the file that uses the include statement.
- Including files is very useful when you want to include the same PHP, HTML, or text on multiple pages of a website.
- It is possible to insert the content of one PHP file into another PHP file with the include or require statement.
  - This happens before the server executes it
- The include and require statements are identical, except upon failure:
  - **require** will produce a fatal error (E_COMPILE_ERROR) and stop the script
  - **include** will only produce a warning (E_WARNING) and the script will continue

When to use require or Include?

- If you want the execution to go on and show users the output, even if the include file is missing, use the **include statement**.
- Otherwise always use the **require statement** to include a key file to the flow of execution.
  - This will help avoid compromising your application's security and integrity, just in-case one key file is accidentally missing.
- Including files saves a lot of work.
  - This means that you can create a standard header, footer, or menu file for all your web pages.
  - When the header needs to be updated, you can only update the header include file.
- **Use require when the file is required by the application.**
- **Use include when the file is not required and application should continue when file is not found.**

PHP include Syntax

- include '*filename*';
  or
  require '*filename*';

```
<!DOCTYPE html>
<html>
<body>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>
<?php include 'footer.php';?>

</body>
</html>
```

Welcome to my home page!

Some text.

Some more text.

Copyright © 1999-2022 W3Schools.com

```
<?php
echo "<p>Copyright &copy; 1999-" . date("Y") . " W3Schools.com</p>";
?>
```

```html
<html>
<body>

<div class="menu">
<?php include 'menu.php';?>
</div>

<h1>Welcome to my home page!</h1>
<p>Some text.</p>
<p>Some more text.</p>

</body>
</html>
```

```php
<?php
echo '<a href="/default.asp">Home</a> -
<a href="/html/default.asp">HTML Tutorial</a> -
<a href="/css/default.asp">CSS Tutorial</a> -
<a href="/js/default.asp">JavaScript Tutorial</a> -
<a href="default.asp">PHP Tutorial</a>';
?>
```

Home - HTML Tutorial - CSS Tutorial - JavaScript Tutorial - PHP 7 Tutorial

Welcome to my home page!

Some text.

Some more text.

PHP File Handling

- File handling is an important part of any web application.
- You often need to open and process a file for different tasks.
- PHP has several functions for manipulating Files:
  - creating, reading, uploading, and editing files.
- File manipulation needs one to e careful as you can do a lot of damage if you do something wrong.
- **Common errors:** editing the wrong file, filling a hard-drive with garbage data, and deleting the content of a file by accident.

PHP readfile() Function

- The **readfile() function** reads a file and writes it to the output buffer.
  - The readfile() function is useful if all you want to do is open up a file and read its contents.
- Assume we have a text file called "webdictionary.txt"
  -
```
AJAX = Asynchronous JavaScript and XML
CSS = Cascading Style Sheets
HTML = Hyper Text Markup Language
PHP = PHP Hypertext Preprocessor
SQL = Structured Query Language
SVG = Scalable Vector Graphics
XML = EXtensible Markup Language
```

```html
<!DOCTYPE html>
<html>
<body>

<?php
echo readfile("webdictionary.txt");
?>

</body>
</html>
```

AJAX = Asynchronous JavaScript and XML CSS = Cascading Style Sheets HTML = Hyper Text Markup Language PHP = PHP Hypertext Preprocessor SQL = Structured Query Language SVG = Scalable Vector Graphics XML = EXtensible Markup Language236

PHP File Open/Read/Close

- A better method to open files is with the **fopen() function**.
  - This function gives you more options than the **readfile() function**.
- The first parameter of fopen() contains the name of the file to be opened and the second parameter specifies in which mode the file should be opened.
- Assume we have a text file called "webdictionary.txt"
  -
```
AJAX = Asynchronous JavaScript and XML
CSS = Cascading Style Sheets
HTML = Hyper Text Markup Language
PHP = PHP Hypertext Preprocessor
SQL = Structured Query Language
SVG = Scalable Vector Graphics
XML = EXtensible Markup Language
```

```
<!DOCTYPE html>
<html>
<body>

<?php
$myfile = fopen("webdictionary.txt", "r") or die("Unable to open file!");
echo fread($myfile,filesize("webdictionary.txt"));
fclose($myfile);
?>

</body>
</html>
```

AJAX = Asynchronous JavaScript and XML CSS = Cascading Style Sheets HTML = Hyper Text Markup Language PHP = PHP Hypertext Preprocessor SQL = Structured Query Language SVG = Scalable Vector Graphics XML = EXtensible Markup Language

Modes	Description
r	**Open a file for read only**. File pointer starts at the beginning of the file
w	**Open a file for write only**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a	**Open a file for write only**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x	**Creates a new file for write only**. Returns FALSE and an error if file already exists
r+	**Open a file for read/write**. File pointer starts at the beginning of the file
w+	**Open a file for read/write**. Erases the contents of the file or creates a new file if it doesn't exist. File pointer starts at the beginning of the file
a+	**Open a file for read/write**. The existing data in file is preserved. File pointer starts at the end of the file. Creates a new file if the file doesn't exist
x+	**Creates a new file for read/write**. Returns FALSE and an error if file already exists

Reading Task: File handling options, Cookies and Sessions

- fread(), fgets(), fwrite() and the fclose() functions
- PHP File Upload, PHP Cookies (setcookie()), PHP Sessions (session_start(), session_unset() and session_destroy())

https://www.w3schools.com/php/php_file_upload.asp

PHP - OOP

- **Object-oriented programming** is about creating objects that contain both data and functions.
  - Alternative to **Procedural programming**: writing procedures or functions that perform operations on the data
- Object-oriented programming has several advantages over procedural programming:
  - OOP is faster and easier to execute
  - OOP provides a clear structure for the programs
  - OOP helps to keep the PHP code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug
    - principle is about reducing the repetition of code
  - OOP makes it possible to create full reusable applications with less code and shorter development time

PHP - Classes and Objects

- **Classes** and **objects** are the two main aspects of object-oriented programming.
  - A **class** is a template for objects
  - An **object** is an instance of a class.
- When the individual objects are created, they inherit all the properties and behaviours from the class, but each object will have different values for the properties.

class	objects
Fruit	Apple Banana Mango

class	objects
Car	Volvo Audi Toyota

Define a Class

- A class is defined by using the **class** keyword, followed by the name of the class and a pair of curly braces ({}).
  - All its properties and methods go inside the braces
    - When the individual objects are created, they inherit all the properties and behaviours from the class, but each object will have different values for the properties.
- Syntax
  - ```php
    <?php
    class Fruit {
      // code goes here...
    }
    ?>
    ```
- Let's declare a class named Fruit consisting of two properties ($name and $color) and two methods **set_name()** and **get_name()** for setting and getting the $name property
    ```

```php
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}
?>
```

In a class, variables are called properties and functions are called methods!

Define Objects

- We can create multiple objects from a class.
- Each object has all the properties and methods defined in the class, but they will have different property values.
- Objects of a class is created using the **new** keyword.
- Syntax:
  - ```
    $apple = new Fruit();
    $banana = new Fruit();
    $apple->set_name('Apple');
    $banana->set_name('Banana');
    ```
    ```

```php
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}

$apple = new Fruit();
$banana = new Fruit();
$apple->set_name('Apple');
$banana->set_name('Banana');

echo $apple->get_name();
echo "<br>";
echo $banana->get_name();
?>
```

- $this keyword refers to the current object, and is only available inside methods.
  - Often used to change the value of the property

```php
<?php
class Fruit {
  // Properties
  public $name;
  public $color;

  // Methods
  function set_name($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
  function set_color($color) {
    $this->color = $color;
  }
  function get_color() {
    return $this->color;
  }
}

$apple = new Fruit();
$apple->set_name('Apple');
$apple->set_color('Red');
echo "Name: " . $apple->get_name();
echo "<br>";
echo "Color: " . $apple->get_color();
?>
```

PHP OOP - Constructor

- A **constructor** allows you to initialize an object's properties upon creation of the object.
- If you create a **__construct()** function, PHP will automatically call this function when you create an object from a class.
- Notice that the construct function starts with two underscores (__)!

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function get_name() {
    return $this->name;
  }
}

$apple = new Fruit("Apple");
echo $apple->get_name();
?>
```

- In this example, using a constructor saves us from calling the set_name() method which reduces the amount of code

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  function get_name() {
    return $this->name;
  }
  function get_color() {
    return $this->color;
  }
}

$apple = new Fruit("Apple", "red");
echo $apple->get_name();
echo "<br>";
echo $apple->get_color();
?>
```

PHP OOP - Destructor

- A destructor is called when the object is destructed or the script is stopped or exited.
- If you create a **__destruct()** function, PHP will automatically call this function at the end of the script.
- Notice that the destruct function starts with two underscores (__)!
- Usually , a **__construct()** function is automatically called when you create an object from a class, and a **__destruct()** function is automatically called at the end of the script
- constructors and destructors helps reducing the amount of code

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name) {
    $this->name = $name;
  }
  function __destruct() {
    echo "The fruit is {$this->name}.";
  }
}

$apple = new Fruit("Apple");
?>
```

```php
<?php
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  function __destruct() {
    echo "The fruit is {$this->name} and the
color is {$this->color}.";
  }
}

$apple = new Fruit("Apple", "red");
?>
```

PHP OOP - Inheritance

- Inheritance in OOP = When a class derives from another class.
- The child class will inherit all the public and protected properties and methods from the parent class.
- In addition, it can have its own properties and methods.
- An inherited class is defined by using the **extends** keyword.

```php
<?php
class Fruit {
  public $name;
  public $color;
  public function __construct($name, $color) {
    $this->name = $name;
    $this->color = $color;
  }
  public function intro() {
    echo "The fruit is {$this->name} and the color is
{$this->color}.";
  }
}

// Strawberry is inherited from Fruit
class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}
$strawberry = new Strawberry("Strawberry", "red");
$strawberry->message();
$strawberry->intro();
?>
```

More content you should know on OOP

- **PHP - Overriding Inherited Methods, PHP OOP – Interfaces, PHP Namespaces, PHP - Inheritance and the Protected Access Modifier, PHP - Interfaces vs. Abstract Classes**

https://www.w3schools.com/php/php_oop_what_is.asp

PHP MySQL Database

- With PHP, you can connect to and manipulate databases.
- MySQL is the most popular database system used with PHP.
- The data in a MySQL database are stored in tables. A table is a collection of related data, and it consists of columns and rows.
- Databases are useful for storing information categorically.
- What is MySQL?: a database system used on the web and runs on a server
  - Ideal for both small and large applications because it is very fast, reliable, and easy to use
  - It uses standard SQL and compiles on a number of platforms
  - It is also free to download and use
  - MySQL is developed, distributed, and supported by Oracle Corporation

Database Queries

- A query is a question or a request.
- We can query a database for specific information and have a recordset returned.
- A company may have a database with the following tables:
  - Employees, Products
  - Customers, Orders
- SQL Query Example:
  - `SELECT LastName FROM Employees`

https://www.w3schools.com/sql/default.asp

PHP Connect to MySQL

- **Open a Connection to MySQ**: Before we can access data in the MySQL database, we need to be able to connect to the server
  - Syntax:
    - ```
      // Create connection
      $conn = mysqli_connect($servername, $username, $password);
      ```
- **Check connection:**
  - ```
    // Check connection
    if ($conn->connect_error) {
      die("Connection failed: " . $conn->connect_error);
    }
    echo "Connected successfully";
    ```
- **Close the Connection:** The connection will be closed automatically when the script ends
  - Syntax:
    - ```
      $conn->close();
      ```
      ```

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}
echo "Connected successfully";
?>
```

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";

// Create connection
$conn = new mysqli($servername, $username, $password);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

// Create database
$sql = "CREATE DATABASE myDB";
if ($conn->query($sql) === TRUE) {
  echo "Database created successfully";
} else {
  echo "Error creating database: " . $conn->error;
}

$conn->close();
?>
```

If you have to use a specific port, add an empty string for the database-name argument, like this: new mysqli("localhost", "username", "password", "", port)

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

// sql to create table
$sql = "CREATE TABLE MyGuests (
id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
firstname VARCHAR(30) NOT NULL,
lastname VARCHAR(30) NOT NULL,
email VARCHAR(50),
reg_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)";

if ($conn->query($sql) === TRUE) {
  echo "Table MyGuests created successfully";
} else {
  echo "Error creating table: " . $conn->error;
}

$conn->close();
?>
```

https://www.w3schools.com/sql/sql_datatypes.asp

```php
<?php
$servername = "localhost";
$username = "username";
$password = "password";
$dbname = "myDB";

// Create connection
$conn = new mysqli($servername, $username, $password, $dbname);
// Check connection
if ($conn->connect_error) {
  die("Connection failed: " . $conn->connect_error);
}

$sql = "INSERT INTO MyGuests (firstname, lastname, email)
VALUES ('John', 'Doe', 'john@example.com')";

if ($conn->query($sql) === TRUE) {
  echo "New record created successfully";
} else {
  echo "Error: " . $sql . "<br>" . $conn->error;
}

$conn->close();
?>
```